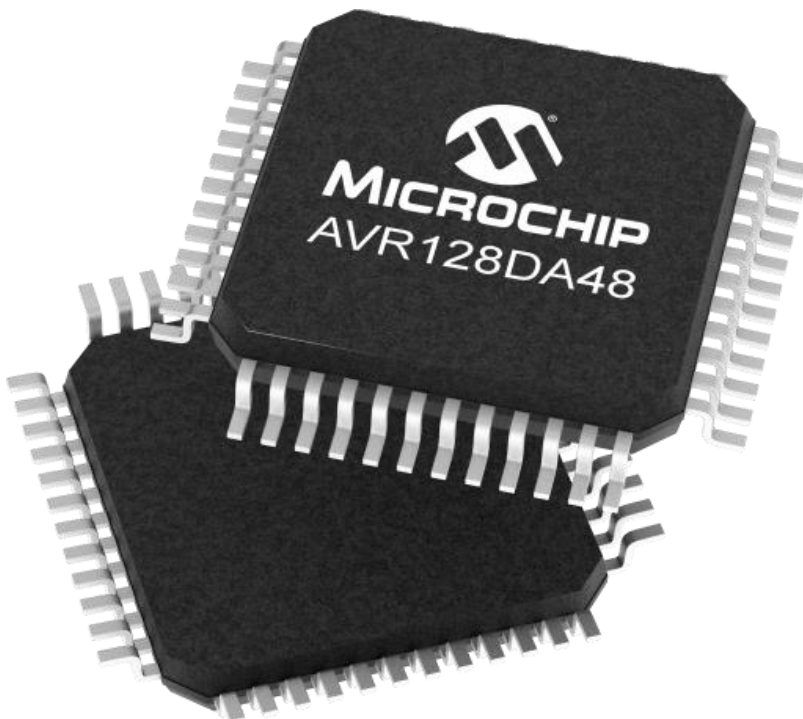
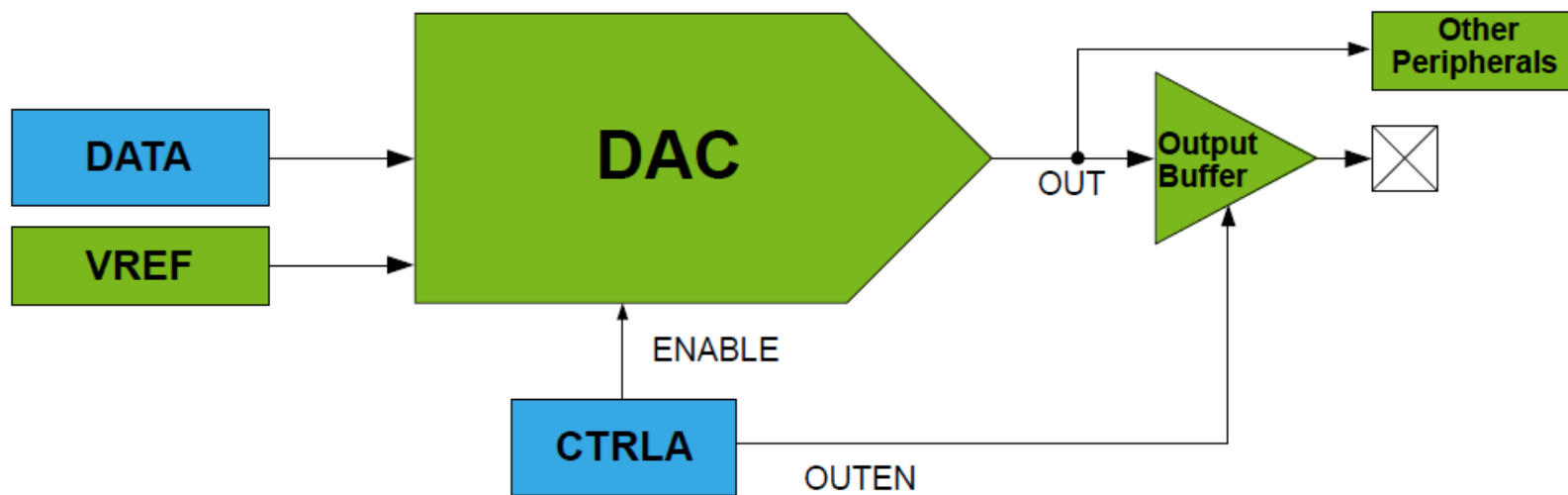


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 19 - DAC Signal Generation, Calibration, and Analog Control



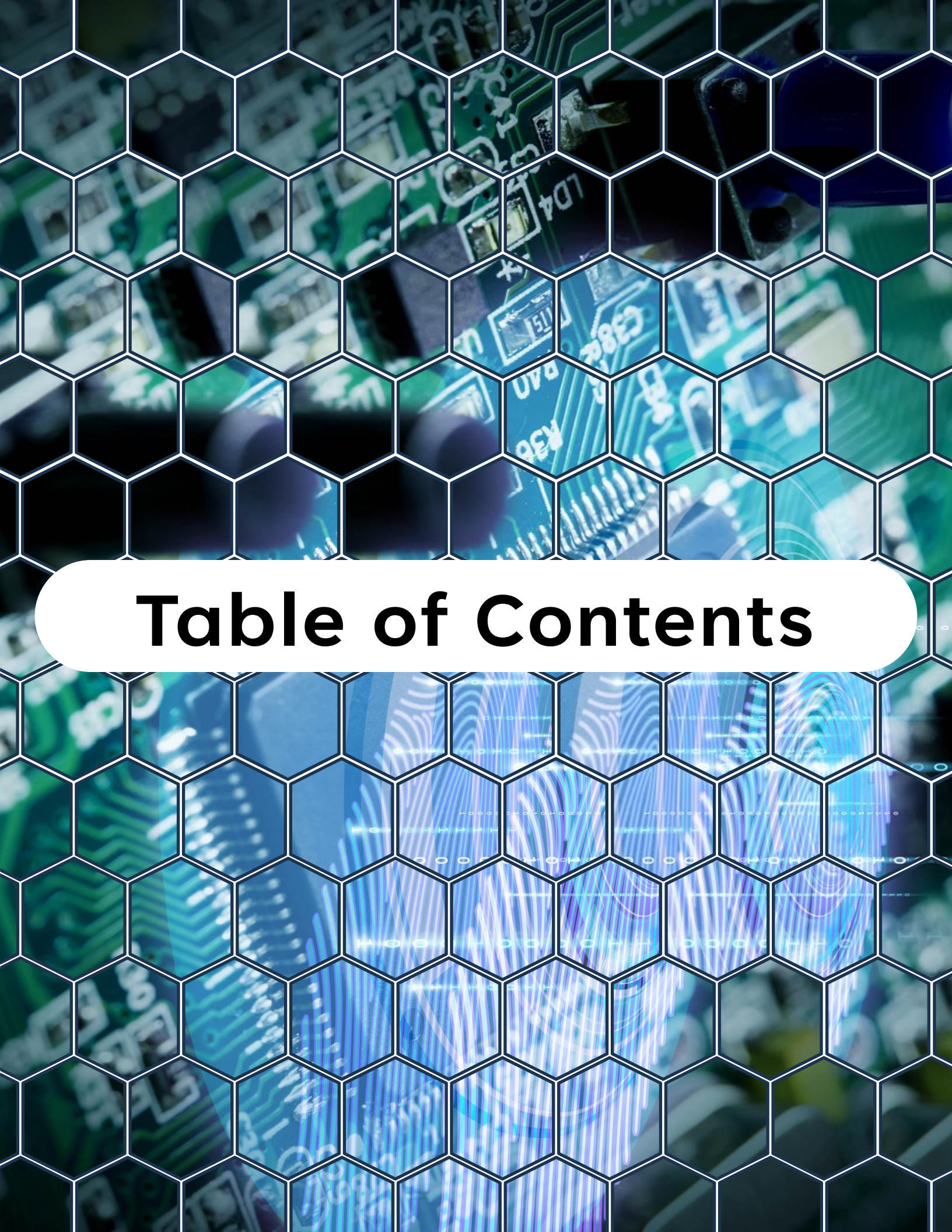
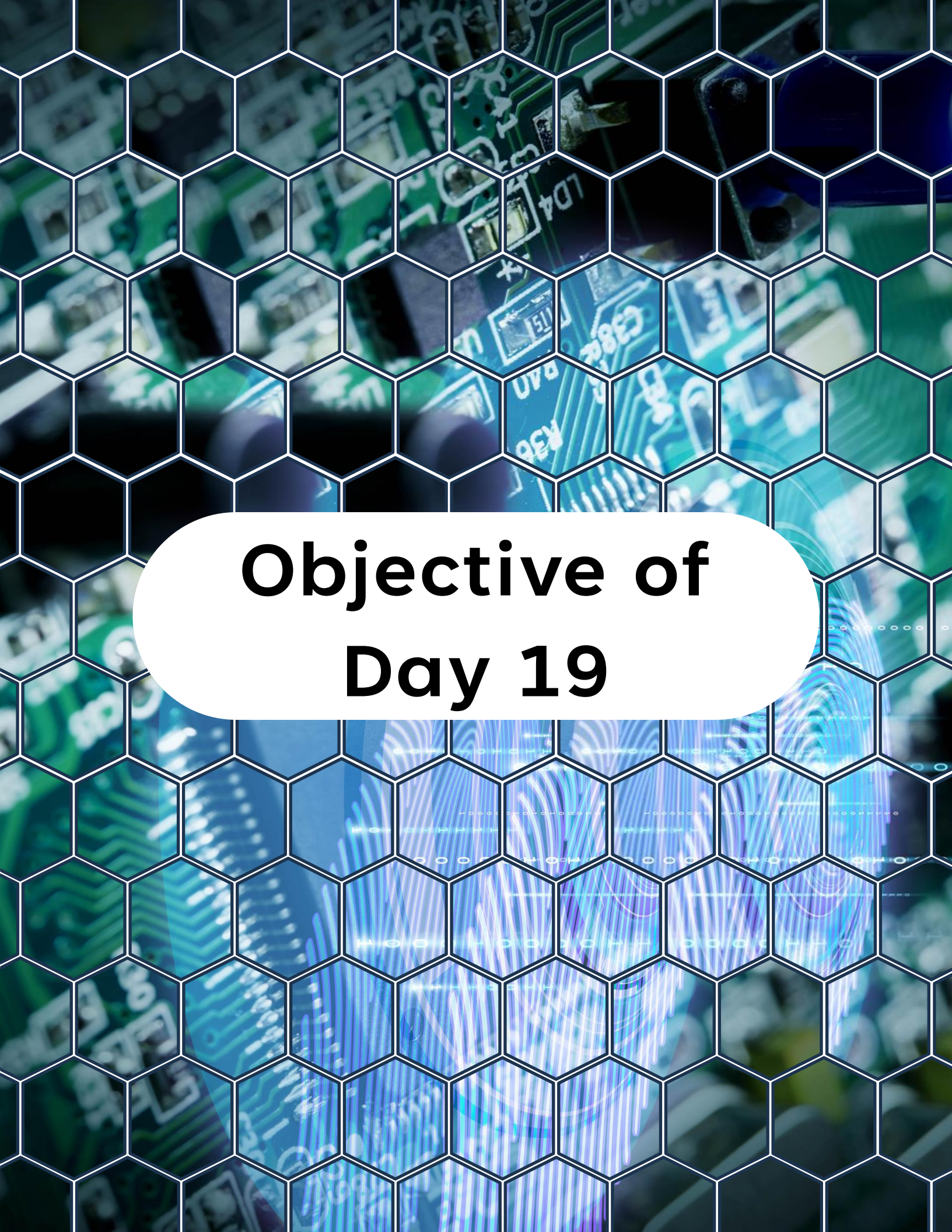


Table of Contents

Table of Contents

Objective of Day 19

1. What Is the DAC?
2. When Do We Use a DAC?
3. DAC Overview (AVR128DA48)
4. Minimal DAC Example - Fixed Voltage Output
5. Generating a Staircase Waveform
6. Generating a Sine Wave (Lookup Table Method)
7. Calibration and Accuracy
8. Using DAC with Analog Comparator
9. Using DAC for Closed-Loop Analog Control
10. Practical Lab for Today
11. What You Should Understand Now
12. Preview of Day 20



Objective of Day 19

Objective of Day 19

Today we step into the analog output domain.

Up to this point, you have measured voltages using the **ADC**, generated digital waveforms using **Timers**, and built interrupt-driven systems. Now we reverse direction. Instead of measuring voltage, we will generate it.

The **AVR128DA48** includes a 10-bit Digital-to-Analog Converter (**DAC**). This peripheral allows you to generate precise analog voltages under firmware control.

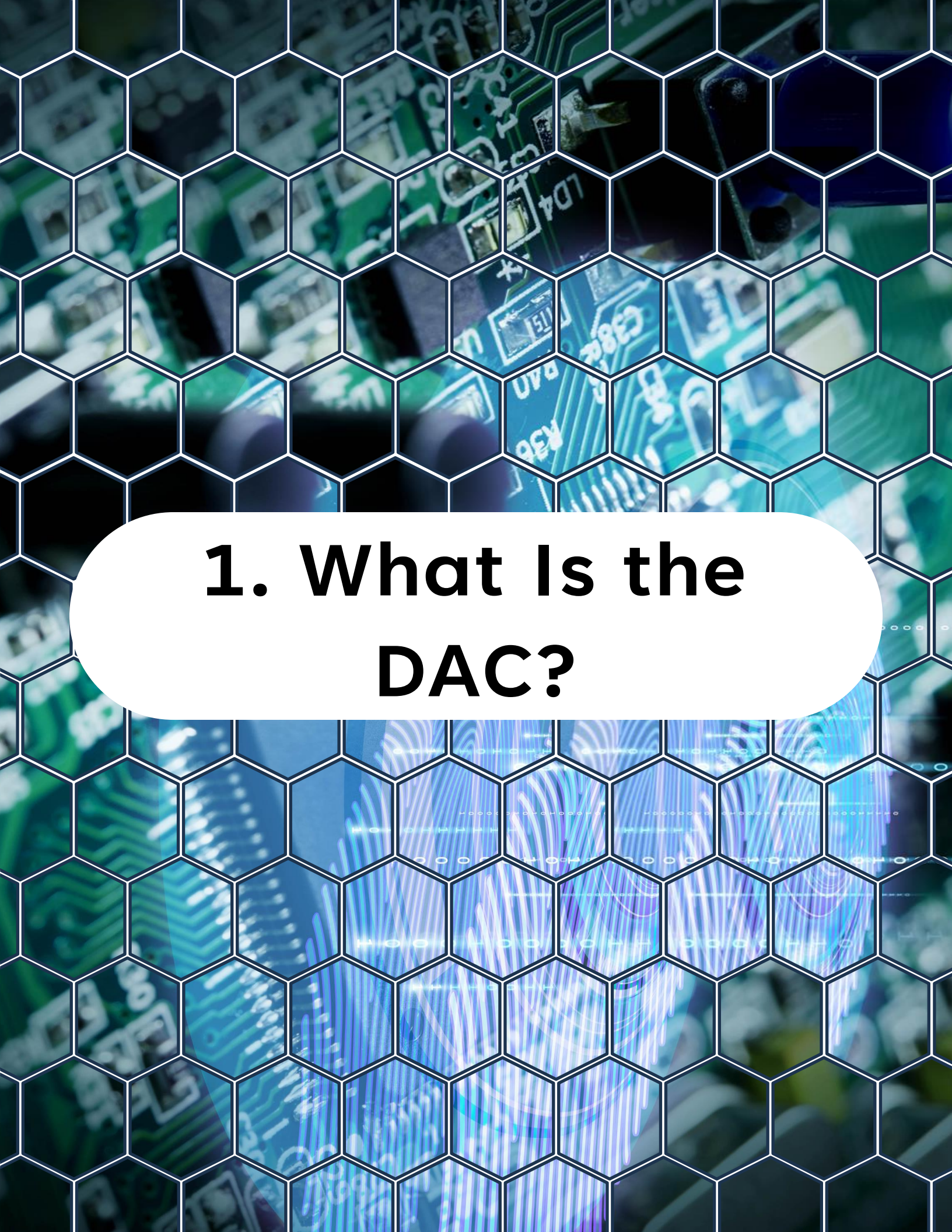
By the end of this day, you will:

- Understand how the DAC works internally
- Configure the DAC at register level

Objective of Day 19

- Generate fixed voltages
- Generate waveform signals (staircase and sine-like)
- Calibrate output levels
- Use the DAC for analog control applications

No frameworks. No middleware. Only registers.



1. What Is the DAC?

1. What Is the DAC?

The DAC converts a digital value into an analog voltage.

On the AVR128DA48:

- Resolution: 10-bit
- Output range: 0V to VREF
- Output pin: typically routed to AIN0 or dedicated DAC output pin
- Can use internal or external reference

The output voltage is:

$$V_{out} = (DAC_VALUE / 1023) * VREF$$

If VREF = 4.096V and DAC_VALUE = 512:

$$V_{out} \approx 2.048V$$

That is deterministic analog control.



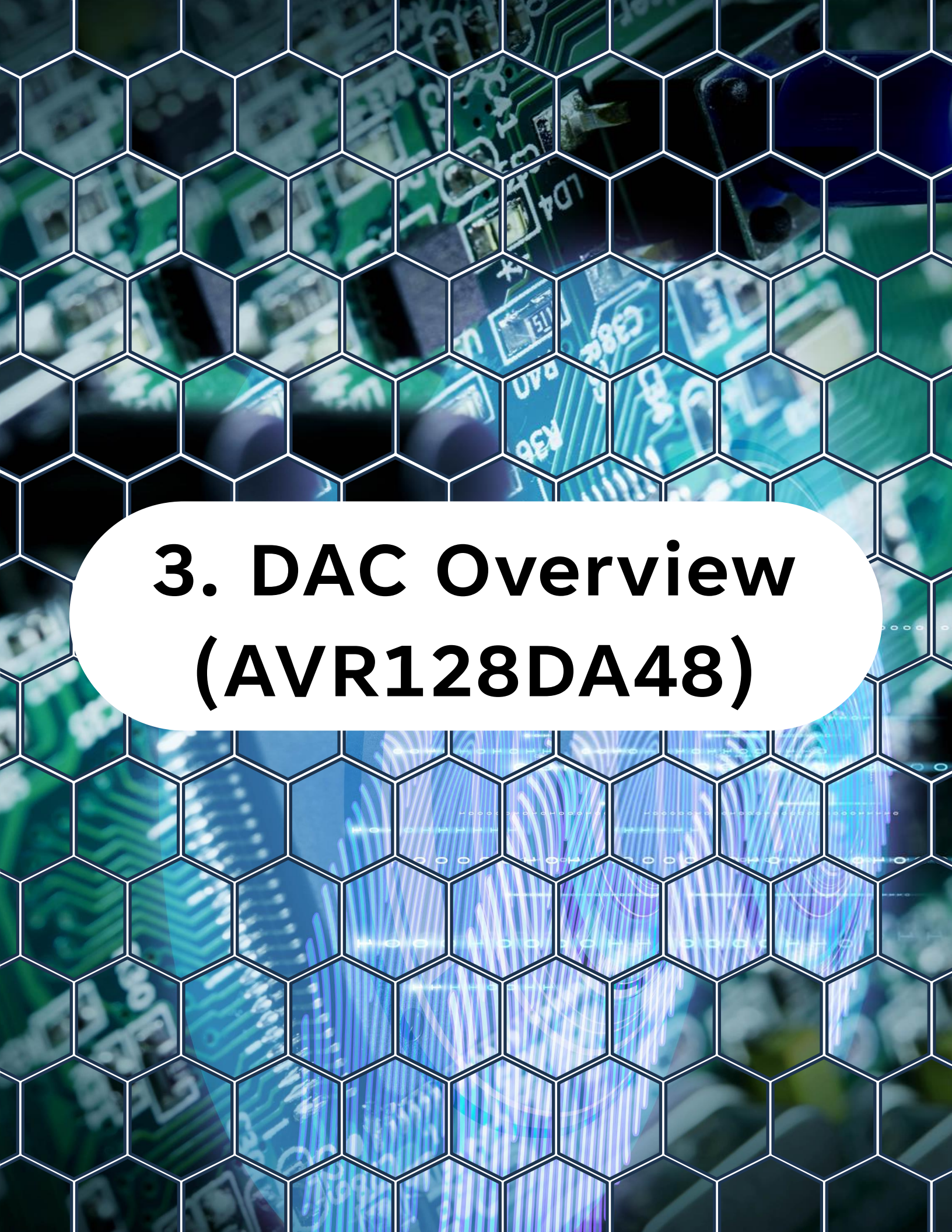
2. When Do We Use a DAC?

2. When Do We Use a DAC?

Common embedded applications:

- Analog bias generation
- Setpoint control for power supplies
- Motor control reference voltage
- Audio tone generation
- Waveform synthesis
- Sensor simulation
- Comparator threshold control

In control systems, the DAC becomes your analog actuator.



3. DAC Overview (AVR128DA48)

3. DAC Overview (AVR128DA48)

Key registers:

- **DAC0.CTRLA**
- **DAC0.DATA** (These bits contain the digital data, which will be converted to an analog voltage.)
- **VREF.DACOREF** (This bit field controls the reference voltage level for DAC0.)

DAC0.CTRLA Important control bits:

- **ENABLE** (Writing a '1' to this bit enables the DAC.)
- **OUTEN** (Writing a '1' to this bit enables the output buffer and sends the OUT signal to a pin.)

3. DAC Overview (AVR128DA48)

VREF.DACOREF Important control bits:

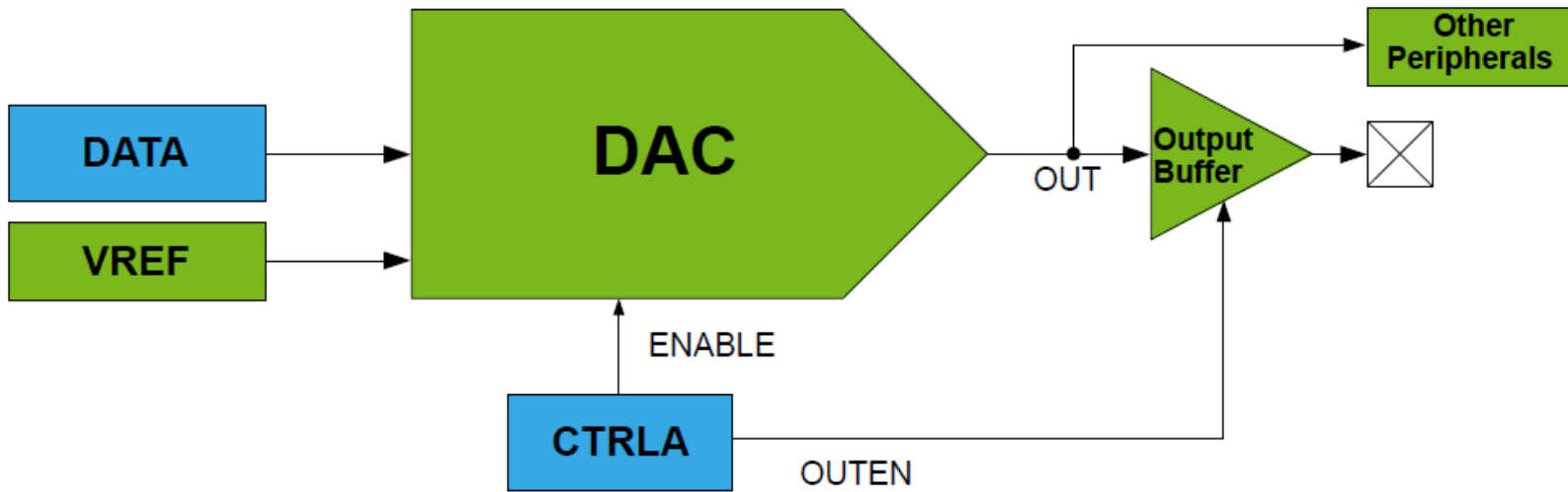
- 1.024V
- 2.048V
- 4.096V
- VDD

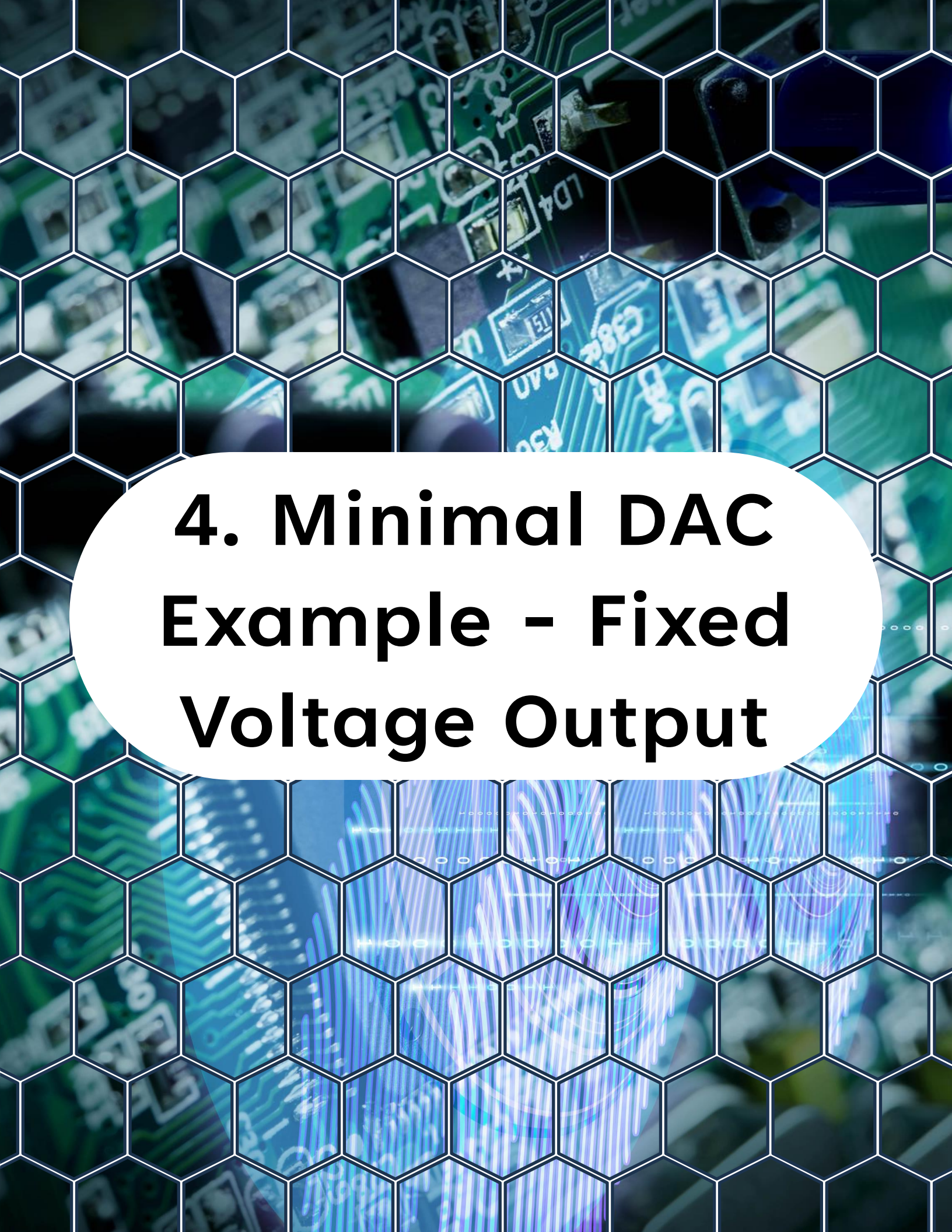
Initialization

1. Select the DAC reference voltage in the Voltage Reference (VREF) peripheral.
2. Enable the output to a pin
3. Then enable DAC.
4. Then write data.

3. DAC Overview (AVR128DA48)

DAC Block Diagram



The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white circular area contains the main title text.

4. Minimal DAC Example - Fixed Voltage Output

4. Minimal DAC Example - Fixed Voltage Output

Goal:

Output approximately 2.0V using internal 4.096V reference.

Voltage Calculation

Target voltage: 2.0V

Reference: 4.096V

$$V_{out} = (DAC_VALUE / 1023) * V_{REF}$$

$$DAC_VALUE = (2.0 / 4.096) * 1023$$

$$DAC_VALUE \approx 500$$

Complete Example

```
1  #include <avr/io.h>
2
3  /**
4   * @brief Initialize DAC0 with internal 4.096V reference.
5   */
```


4. Minimal DAC Example - Fixed Voltage Output

Complete Example

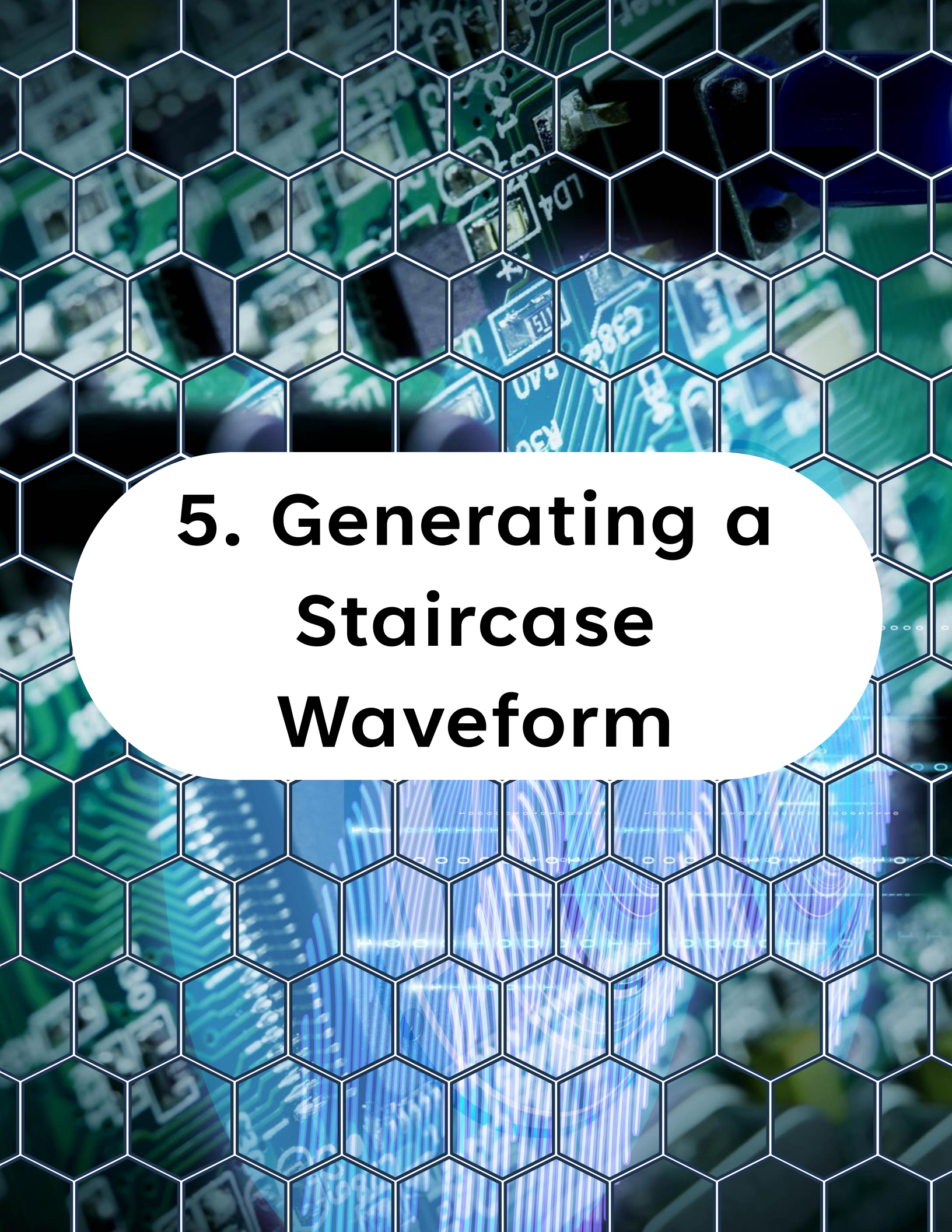
```
1  #include <avr/io.h>
2
3  /**
4   * @brief Initialize DAC0 with internal 4.096V reference.
5   */
6  static void DAC0_init(void) {
7      /* Select 4.096V reference for DAC */
8      VREF.DAC0REF = VREF_REFSEL_4V096_gc;
9
10     /* Enable DAC output buffer and DAC */
11     DAC0.CTRLA = DAC_ENABLE_bm | DAC_OUTEN_bm;
12 }
13
14 /**
15  * @brief Set DAC output value.
16  *
17  * @param value 10-bit value (0-1023)
18  */
19 static void DAC0_set(uint16_t value) {
20     DAC0.DATA = value;
21 }
22
23 int main(void) {
24
25     DAC0_init();
26
27     /* Output ~2.0V */
28     DAC0_set(500);
29 }
```

4. Minimal DAC Example - Fixed Voltage Output

```
14 /**
15  * @brief Set DAC output value.
16  *
17  * @param value 10-bit value (0-1023)
18  */
19 static void DAC0_set(uint16_t value) {
20     DAC0.DATA = value;
21 }
22
23 int main(void) {
24
25     DAC0_init();
26
27     /* Output ~2.0V */
28     DAC0_set(500);
29
30     while (1)
31     {
32         /* Steady output */
33     }
34 }
```

Measure the output with a multimeter or oscilloscope.

You should see approximately 2.0V.



5. Generating a Staircase Waveform

5. Generating a Staircase Waveform

Now let us generate a slow staircase waveform.

This is useful for:

- Testing ADC
- Testing comparator thresholds
- Debugging analog circuits

Concept

Increment DAC value in a loop.

Add delay between steps.

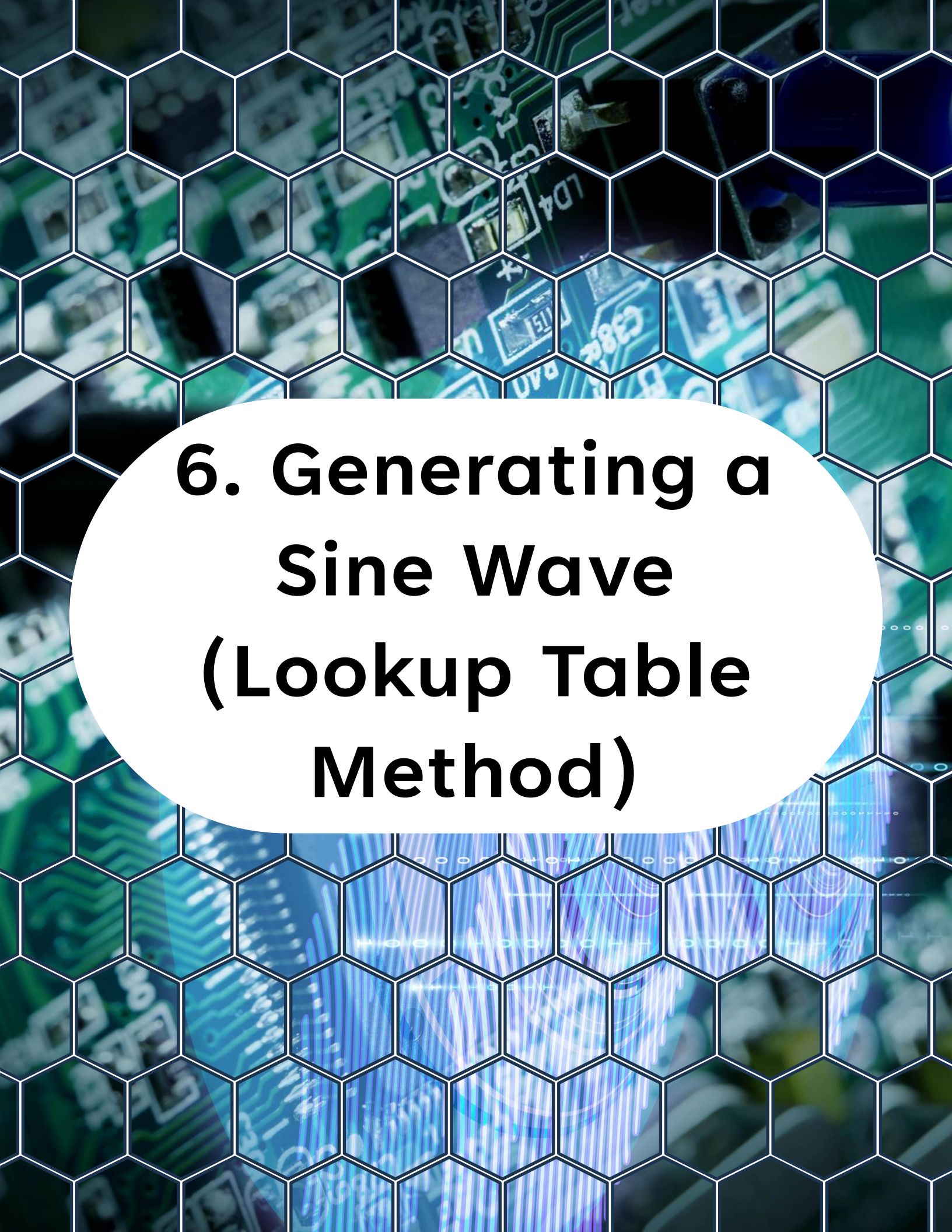
Example

```
1  /**
2   * @brief Generate a simple staircase waveform using DAC.
3   */
4  int main(void)
5  {
6      uint16_t value = 0;
7
```

5. Generating a Staircase Waveform

```
1  /**
2   * @brief Generate a simple staircase waveform using DAC.
3   */
4  int main(void)
5  {
6      uint16_t value = 0;
7
8      DAC0_init();
9
10     while (1)
11     {
12         DAC0_set(value);
13
14         value += 32;
15
16         if (value > 1023)
17         {
18             value = 0;
19         }
20
21         for (volatile uint32_t i = 0; i < 50000; i++)
22         {
23             /* crude delay */
24         }
25     }
26 }
```

Observe the stepped ramp on an oscilloscope.
You just built a waveform generator.



6. Generating a Sine Wave (Lookup Table Method)

6. Generating a Sine Wave (Lookup Table Method)

For precise waveform synthesis, use a lookup table.

Approach

- Create sine table (e.g., 64 samples)
- Periodically update DAC
- Control update rate to set frequency

This is where TCB interrupts or RTC become powerful.

Example Generating a Sine Wave Using DAC:

```
1 #include <avr/io.h>
2 #include <avr/interrupt.h>
3
4 #define SINE_SAMPLES 32
5
6 /* 32-point sine table (centered at 512, amplitude ~400) */
7 static const uint16_t sine_table[SINE_SAMPLES] =
8 {
9     512, 500, 465, 426, 384, 340, 295, 250,
10     205, 160, 115, 71, 28, 15, 3, 0,
11     0, 3, 15, 28, 71, 115, 160, 205,
12     250, 295, 340, 384, 426, 465, 500, 512
```

6. Generating a Sine Wave (Lookup Table Method)

```
6  /* 32-point sine table (centered at 512, amplitude ~400) */
7  static const uint16_t sine_table[SINE_SAMPLES] =
8  {
9      512, 590, 665, 736, 800, 856, 902, 937,
10     960, 972, 972, 960, 937, 902, 856, 800,
11     736, 665, 590, 512, 434, 359, 288, 224,
12     168, 122, 87, 64, 52, 52, 64, 87
13 };
14
15 static volatile uint8_t sine_index = 0;
16
17 /**
18  * @brief Initialize DAC0 with 4.096V internal reference.
19  */
20 static void DAC0_init(void)
21 {
22     /* Select 4.096V reference for DAC */
23     VREF.DAC0REF = VREF_REFSEL_4V096_gc;
24
25     /* Enable DAC output and DAC itself */
26     DAC0.CTRLA = DAC_ENABLE_bm | DAC_OUTEN_bm;
27 }
28
29 /**
30  * @brief Initialize TCB0 in periodic interrupt mode.
31  *
32  * Generates interrupt at approximately 1 kHz.
33  */
34 static void TCB0_init(void)
35 {
36     /* Set compare value for interrupt frequency */
37     TCB0.CCMP = 12000; /* 12 MHz / 12000 = 1 kHz */
38
39     /* Enable interrupt */
40     TCB0.INTCTRL = TCB_CAPT_bm;
```

6. Generating a Sine Wave (Lookup Table Method)

```
29 /**
30  * @brief Initialize TCB0 in periodic interrupt mode.
31  *
32  * Generates interrupt at approximately 1 kHz.
33  */
34 static void TCB0_init(void)
35 {
36     /* Set compare value for interrupt frequency */
37     TCB0.CCMP = 12000; /* 12 MHz / 12000 = 1 kHz */
38
39     /* Enable interrupt */
40     TCB0.INTCTRL = TCB_CAPT_bm;
41
42     /* Periodic interrupt mode, clock = CLK_PER/2 */
43     TCB0.CTRLA = TCB_CLKSEL_DIV2_gc | TCB_ENABLE_bm;
44 }
45
46 /**
47  * @brief TCB0 interrupt service routine.
48  *
49  * Outputs next sine sample to DAC.
50  */
51 ISR(TCB0_INT_vect)
52 {
53     /* Clear interrupt flag */
54     TCB0.INTFLAGS = TCB_CAPT_bm;
55
56     /* Output next sine sample */
57     DAC0.DATA = sine_table[sine_index];
58
59     sine_index++;
60
61     if (sine_index >= SINE_SAMPLES)
62     {
63         sine_index = 0;
```


6. Generating a Sine Wave (Lookup Table Method)

```
51 ISR(TCB0_INT_vect)
52 {
53     /* Clear interrupt flag */
54     TCB0.INTFLAGS = TCB_CAPT_bm;
55
56     /* Output next sine sample */
57     DAC0.DATA = sine_table[sine_index];
58
59     sine_index++;
60
61     if (sine_index >= SINE_SAMPLES)
62     {
63         sine_index = 0;
64     }
65 }
66
67 /**
68  * @brief Main entry point.
69  */
70 int main(void)
71 {
72     DAC0_init();
73
74     TCB0_init();
75
76     /* Enable global interrupts */
77     sei();
78
79     while (1)
80     {
81         /* All waveform generation handled in ISR */
82     }
83 }
```

6. Generating a Sine Wave (Lookup Table Method)

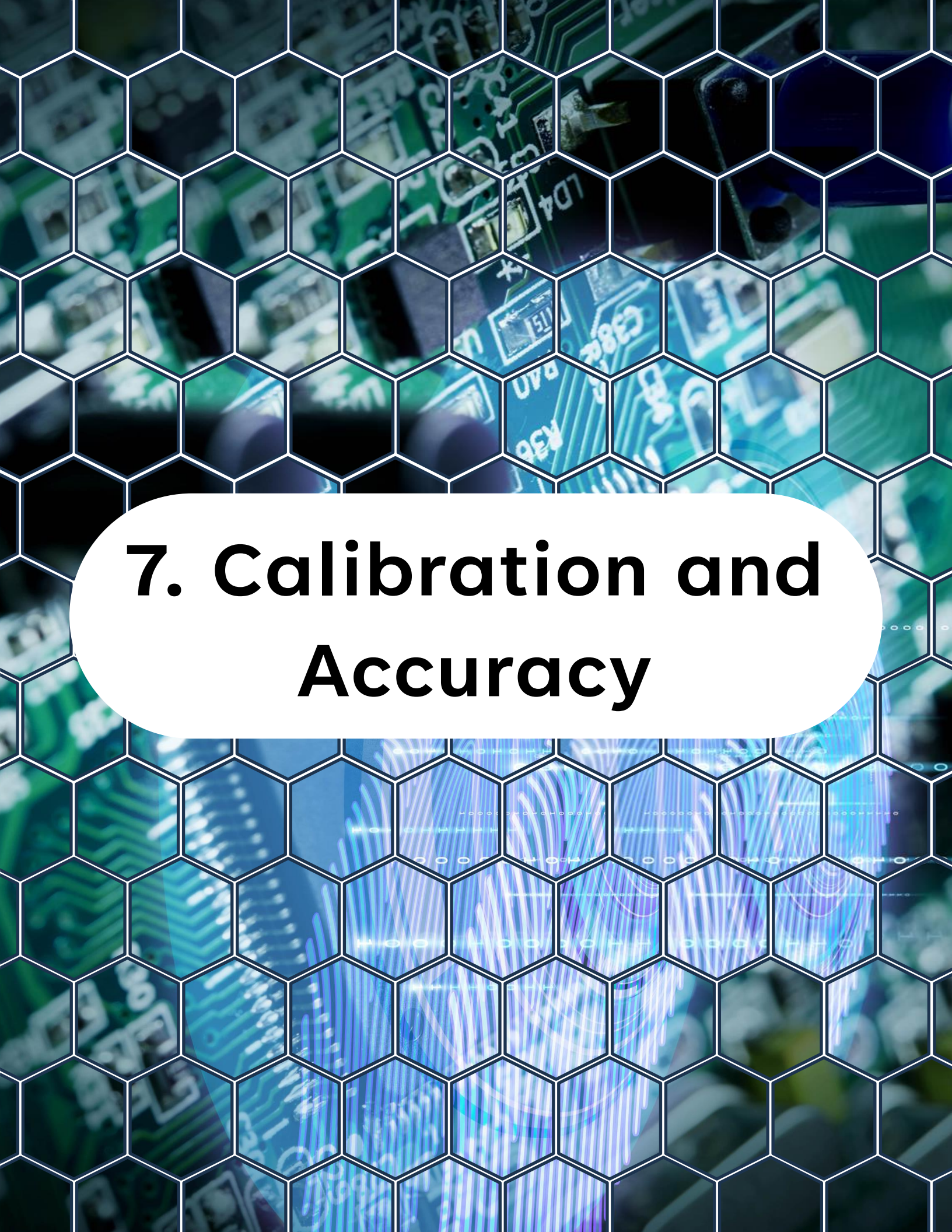
What You Should See

On an oscilloscope connected to the DAC output pin:

- Smooth sine-like waveform
- Approximately 31 Hz
- Peak-to-peak $\approx 3.2V$
- Centered around $\sim 2.048V$

It will not be mathematically perfect.

But it is deterministic, periodic, and hardware-driven.



7. Calibration and Accuracy

7. Calibration and Accuracy

Real DAC output is not perfect.

You must consider:

- Reference tolerance
- Temperature drift
- Output buffer offset
- VDD variation (if VDD used as reference)

Calibration Strategy

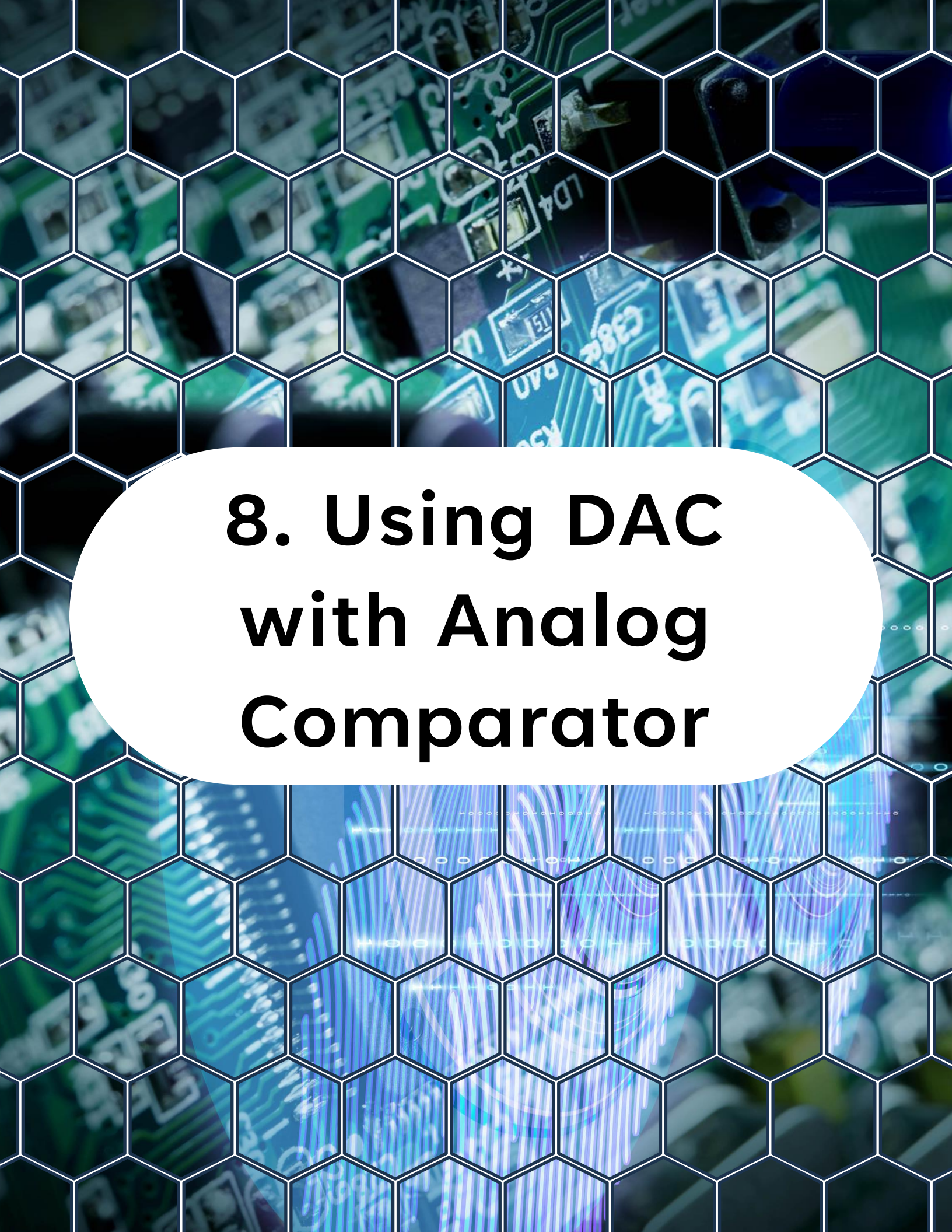
1. Measure actual output at two known DAC values.
2. Calculate slope correction.
3. Apply correction factor in software.

7. Calibration and Accuracy

Example correction model:

$$\text{corrected_value} = (\text{target_voltage} / \text{measured_vref}) * 1023$$

Professional embedded systems always calibrate analog outputs.



8. Using DAC with Analog Comparator

8. Using DAC with Analog Comparator

Powerful integration:

- DAC -> Internal AC reference
- AC compares external signal against DAC threshold

Use case:

- Adjustable threshold detection
- Programmable voltage trip
- Window comparator systems

You now have dynamic analog thresholds without external resistors.

This is extremely powerful in power electronics and sensing systems.



9. Using DAC for Closed-Loop Analog Control

9. Using DAC for Closed-Loop Analog Control

In control systems:

- ADC reads feedback
- Controller computes error
- DAC outputs correction signal

That is a classic digital control loop.

For example:

- Control a buck converter
- Set reference for op-amp
- Generate audio amplitude control
- Adjust motor drive bias

This bridges digital firmware and analog hardware.



10. Practical Lab for Today

10. Practical Lab for Today

1. Output fixed 1.5V, measure it.
2. Sweep from 0V to VREF in 100 steps.
3. Generate 1Hz triangle wave.
4. Feed DAC output into ADC input and measure loopback.
5. Use DAC as comparator threshold and detect crossing.

Do not simulate. Use real hardware.

Measure everything.



11. What You Should Understand Now

11. What You Should Understand Now

You should now be comfortable with:

- Configuring VREF for DAC
- Enabling DAC output
- Writing 10-bit output values
- Generating analog waveforms
- Understanding DAC resolution limits
- Performing basic calibration
- Integrating DAC with comparator

You are no longer writing firmware that only reads the world.

You are now controlling it.



12. What Comes Next

12. What Comes Next

Tomorrow we integrate:

Analog Comparator (AC) + DAC + EVSYS

We will build:

Hardware threshold detection with zero CPU load.

That is where embedded design becomes elegant.

See you on Day 20.

You can find all source code, examples, and updates for this course in the GitHub repository, make sure to visit it, explore the files, and clone it so you can follow along hands-on.

<https://github.com/god233012yamil/Bare-Metal-Embedded-Systems-with-AVR128DA48-Course>